

# Learning Report

Tim Chadwick

July 2026

## 1 Week 1

### 1.1 Python file processing

Task goal: create a script that takes an input .txt file, cleans it and outputs a .csv file with all words and how frequently they appear.

### 1.2 Dataset processing using pandas

The dataset contains information on fortune 1000 companies, including rank, revenue, profit etc. It was cleaned by removing non-numeric data e.g. CEO name, and removing symbols from numeric data so it can be analysed. The following table contains a small amount of information on dataset size and mean values. The missing values were not removed as they mostly correspond to companies that are new or were not in the fortune 1000 list the previous year.

| Dataset size           | Feature count | Missing value count | Mean Revenue | Mean Profit |
|------------------------|---------------|---------------------|--------------|-------------|
| 1000 rows x 10 columns | 10            | 402                 | 14378.202400 | 1371.597015 |

### 1.3 Creating plots

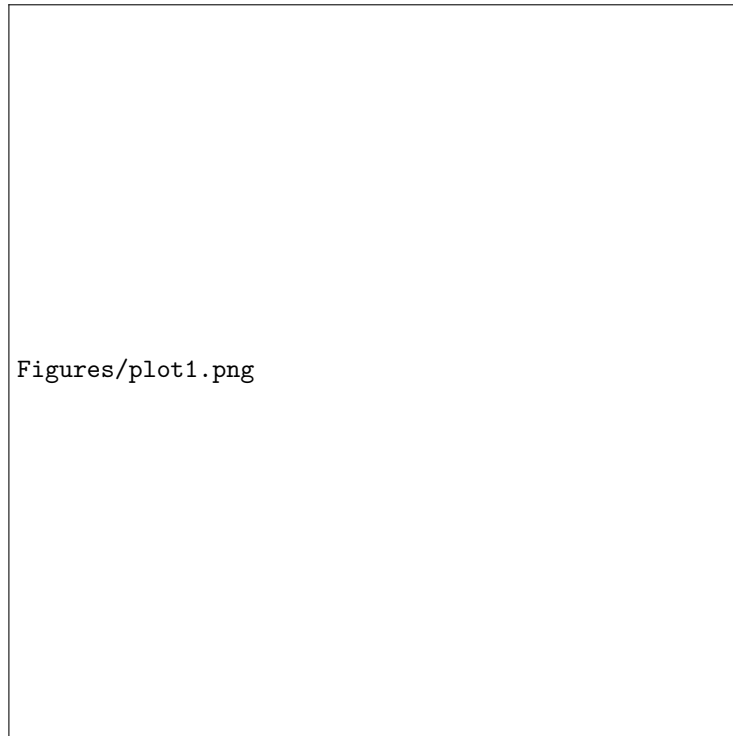


Figure 1: Revenue vs no. employees



Figure 2: Profit vs no. employees

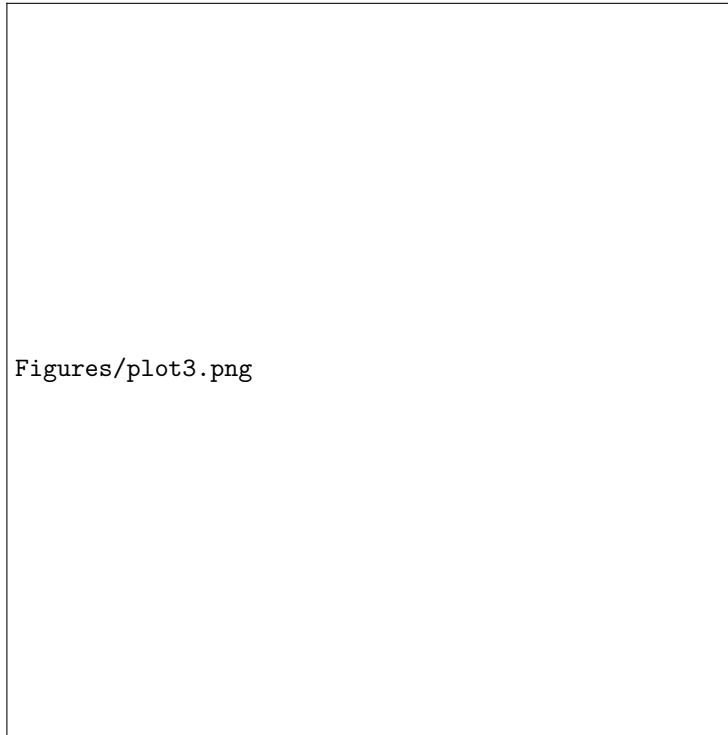


Figure 3: Rank increase vs revenue change %

## 1.4 Results Analysis

Figure 1 shows a positive relationship between the number of employees and company revenue. In general, larger companies tend to generate higher revenues, although there is considerable variation among companies of similar size. A small number of very large companies appear as outliers with exceptionally high employee counts and revenues.

Figure 2 indicates that company profit does not increase as consistently with employee numbers as revenue does. While some larger companies achieve high profits, others generate relatively modest profits despite employing many people, suggesting that profitability depends on factors beyond company size alone.

Figure 3 demonstrates a clear positive relationship between revenue change and changes in Fortune 1000 ranking. Companies with higher percentage increases in revenue generally experience larger improvements in rank, whereas companies with declining revenue tend to lose ranking positions. Although the overall trend is strong, several outliers indicate that ranking can also be influenced by other business factors.

Overall, the visualisations suggest that employee count is positively associated with revenue, but is a weaker predictor of profit. Revenue growth appears to be strongly related to improvements in company ranking.

## 2 Week 2

### 2.1 ML workflow

Definitions:

Feature: Independent variables used by the model to make predictions. In the fortune1000 dataset, these are rank, revenue, employees, etc.

Label: What we are trying to predict, the output of the ML model. In this case, it is profit.

Training set: The set of data that we give the regression model to train it so that it can make predictions. Usually about 80% of available data.

Test set: The set of data we save for after training, to test the model on unseen data and compare the predictions to exact values and evaluate the performance.

Loss: A numerical measure of the accuracy of the model. During training the model adjusts parameters in order to minimise the loss. Common measures of loss are mean absolute error (average difference between prediction and true value), and mean squared error (average of square of difference between prediction and true value), which penalises larger prediction errors more heavily.

### 2.2 Linear regression: Fortune 1000 model

Table 1: Fortune 1000 model results

| Metric | Value       |
|--------|-------------|
| MAE    | 936.556     |
| MSE    | 7290865.704 |
| RMSE   | 2700.160    |
| $R^2$  | 0.642       |

The model shows a fairly strong correlation between company profit and the input features, shown by the  $R^2$  value of 0.642. Despite this, the MAE of 937 is quite high considering the mean profit is 1371, and the RMSE of 2700 shows the model makes some quite large prediction errors as these are penalised more heavily by the MSE. Performance could likely be improved by including data from previous years, including more features, or normalising the features so be close to each other in magnitude.

### 2.3 Classifiers

#### 2.3.1 Classification metrics:

In the following, TP stands for true positives; the number of actual positives correctly identified as positive by the model. TN stands for true negatives; the number of actual negatives classified as negative. FP stands for false positives;

actual negatives that are incorrectly classified as positive by the model. In the following example using the Titanic dataset, this would mean the model predicts a passenger survived when they actually did not. FN stands for false negatives; the number of actual positives that the model incorrectly classifies as negative. In the example of the Titanic, this would mean the model predicted the passenger died when they actually survived.

**Accuracy:** The proportion of correct classifications, both positive and negative.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

**Recall:** The proportion of actual positives that are correctly classified as true positives, aka true positive rate (TPR).

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

**Precision:** The proportion of the model's positive classifications that are actual positives.

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

**F1 score:** The harmonic mean of precision and recall.

$$F1 = 2 \times \frac{precision \times recall}{precision + recall} = \frac{2TP}{2TP + FP + FN} \quad (4)$$

### 2.3.2 Training classifiers on Titanic dataset

|           | Logistic Regression | Decision Tree |
|-----------|---------------------|---------------|
| Accuracy  | 0.815642            | 0.765363      |
| Precision | 0.803030            | 0.746032      |
| Recall    | 0.726027            | 0.643836      |
| F1-score  | 0.762590            | 0.691176      |

This exercise involved training 2 different classification models on the same dataset, in this case data on the people who died on the Titanic in order to compare the classification metrics. The results show that the logistic regression model performs better by every metric. Both models have recall as the lowest metric (other than accuracy), and precision as the highest, which means they are better at avoiding false positives (incorrectly predicting that someone died), than false negatives (incorrectly predicting that someone survived).

### 2.3.3 Cross-validation

When using cross-validation on the same data, the mean accuracy over all 5 folds was lower than just using the original train/test split, though 1 of the folds had a higher accuracy. Usually this is a more accurate number for accuracy as the model is trained over the entire dataset instead of a particular training split.

## 2.4 Neural networks

### 2.4.1 Pytorch training algorithm

1. Initialise the model:  
Create the neural network architecture including layers, activation functions and forward pass, with randomly initialised weights and biases.
2. Define loss function:  
Select a loss function e.g. BCEWithLogitsLoss to measure how well the model is performing.
3. Define the optimiser:  
Select an optimiser e.g. Adam or SGD which updates the model's weights and biases during training.
4. Load training data:  
Import or generate training data, and create the train/test split.
5. Train model over a number of epochs:
  - (a) Set model to training mode.
  - (b) Complete a forward pass.
  - (c) Calculate loss using selected function
  - (d) Zero previous gradients
  - (e) Perform backpropagation which calculates the gradient with respect to every parameter at each step
  - (f) Update the model using the optimiser
6. Evaluate the model:  
Periodically during training or after training has finished, set model to evaluate mode, evaluate the model on training and test data using metrics such as accuracy.
7. Record results:  
Store training and testing metrics at each evaluation point. Store in dataframe and export to csv if required.

### 2.4.2 Example blob classifier neural network

|   | Epoch | Train loss | Train accuracy | Test loss | Test accuracy |
|---|-------|------------|----------------|-----------|---------------|
| 0 | 0     | 0.660000   | 49.125000      | 0.628000  | 61.500000     |
| 1 | 10    | 0.473000   | 96.750000      | 0.457000  | 96.500000     |
| 2 | 20    | 0.282000   | 98.125000      | 0.274000  | 98.500000     |
| 3 | 30    | 0.166000   | 99.000000      | 0.162000  | 99.000000     |
| 4 | 40    | 0.110000   | 99.375000      | 0.107000  | 99.000000     |
| 5 | 50    | 0.081000   | 99.625000      | 0.079000  | 99.500000     |
| 6 | 60    | 0.064000   | 99.625000      | 0.062000  | 100.000000    |
| 7 | 70    | 0.053000   | 99.500000      | 0.051000  | 100.000000    |
| 8 | 80    | 0.046000   | 99.625000      | 0.044000  | 100.000000    |
| 9 | 90    | 0.040000   | 99.625000      | 0.038000  | 100.000000    |

## 2.5 Image classifier

### 2.5.1 Simple image classifier from scratch

The following are the results of a simple image classifier trained on the MNIST dataset to classify images as digits from 0-9. It is a neural network but contains no non-linearities between the layers so is very simple.

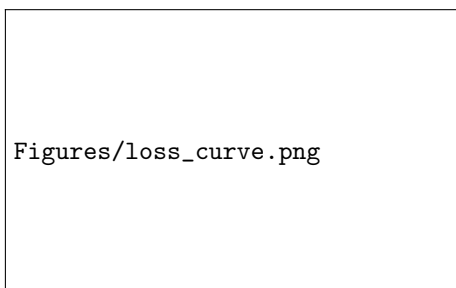
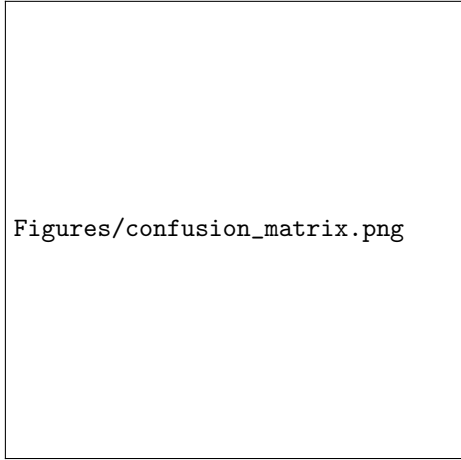
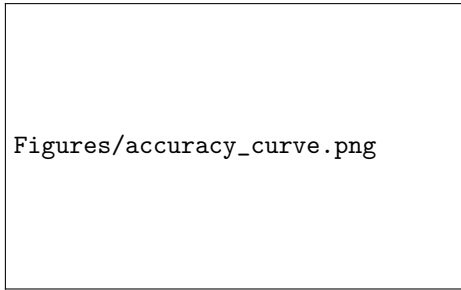


Figure 4: Testing and training loss over time



Figures/confusion\_matrix.png

Figure 5: Confusion matrix



Figures/accuracy\_curve.png

Figure 6: Testing and training accuracy over time

### 2.5.2 Image classifier from pre-trained model

I used the resnet18 model which has been pretrained on the ImageNet database to classify images, and adapted it to take the MNIST dataset I was using. The code was unable to run on my cpu so I have no comparison figures, but because the images being classified are very simple, and the original model performed well it would be unlikely to offer a big increase in performance.

## 3 Week 3

### 3.1 Transfer learning

#### 3.1.1 Definitions

Domain adaptation, DA:

A type of transfer learning where you take a model that has been pre-trained on a dataset (source domain), and you adapt the model to work on the desired dataset (target domain).

Domain generalisation, DG:

A type of transfer learning where a model is pre-trained on many source domain to work on multiple unseen datasets.

Source domain:

The dataset that the initial model was originally trained on before transfer learning starts.

Target domain:

The new dataset/environment that you want to adapt the model to work on.

Backbone:

The main part of a neural network that extracts features, excludes the final classifying layer.

### 3.1.2 Domain adaptation example, dan.py

dan.py is a domain adaptation code used for image classification, which takes the architecture of resnet18 image classifier and adapts it to the target domain set by the user.

```
image.png
```

In this dan.py example, the dataset used is defined here in line 200, with the default being Office31. The source and target domain are not predefined, but the user can select between Amazon, Webcam, and Dslr.

```
image1.png
```

This is the line that controls the train/test split. The actual split is handled by the `utils.get_dataset` function.

```
image2.png
```

The above line creates the backbone which is again handled by a function in `utils`, `get_model`.

```
image3.png
```

Above is the code which defines the architecture, by default dan.py uses resnet18's architecture.

The metric used in this code is accuracy, which is also imported from `utils`.